

1. What is an Operating System? List the services that an operating system provides to its users.

Definition of Operating System:

An Operating System (OS) is a system software that acts as an intermediary between the user and computer hardware. Its primary objective is to provide an environment in which users can execute programs conveniently and efficiently.

An operating system controls and coordinates the use of hardware resources such as CPU, memory, storage devices, and I/O devices among various application programs and users. It also acts as a resource allocator and a control program.

The operating system can also be defined as the program that is running at all times on the computer, called the kernel.

Operating System Services

Operating systems provide services for the convenience of users and programmers and to make program execution easier. The major services provided by an operating system are:

1. User Interface (UI)

The operating system provides an interface between the user and the computer system.

Types of user interfaces:

- Command Line Interface (CLI)- uses text commands
- Batch Interface
- Graphical User Interface (GUI) is a window system with a pointing device (i.e. mouse) to direct I/O

2. Program Execution

The operating system loads programs into main memory, executes them, and handles their normal or abnormal termination.

3. I/O Operations

The operating system manages input and output operations requested by running programs involving files or I/O devices. ex: recording to a CD or DVD

drive

4. File System Manipulation

The operating system provides facilities to:

- Create and delete files and directories
- Read and write files
- Search files
- Manage file permissions and ownership

5. Communication

Processes need to exchange information with one another.

Communication can be implemented through:

- **Shared Memory**-two or more processes read and write to a shared section of memory.
- **Message Passing**-the packets of information in predefined formats are moved between processes by the operating system.

6. Error Detection

The operating system continuously monitors and detects errors related to:

- **CPU and memory hardware errors such as a memory error**
- **I/O devices errors such as a parity error on disk, a connection failure on a network, or lack of paper in the printer**
- **User programs errors such as an arithmetic overflow**

It takes appropriate actions to maintain system consistency.

7. Resource Allocation

When there are multiple users or multiple jobs running at the same time, resources must be allocated to each of them.

The operating system allocates resources such as:

- **CPU time**
- **Main memory**
- **File storage**
- **I/O devices**

among multiple users and processes efficiently.

8. Accounting

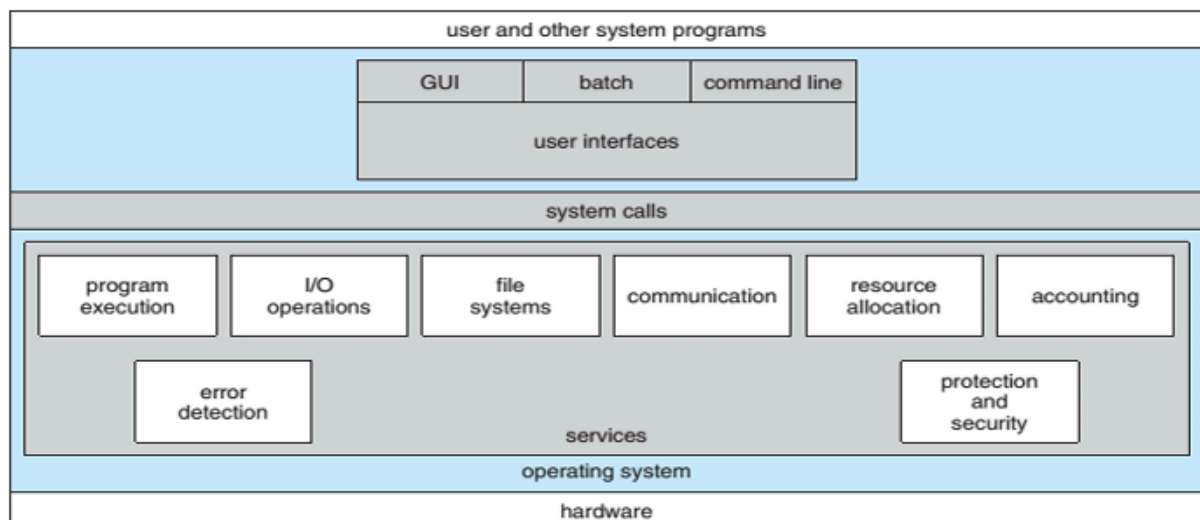
The operating system keeps track of resource usage by users and processes for billing, performance analysis, and system improvement.

9. Protection and Security

- Protection controls access to system resources and prevents interference among processes.
- Security protects the system against unauthorized access, viruses, identity theft, and other attacks through authentication mechanisms such as usernames and passwords.

Conclusion

Thus, an operating system acts as an interface between users and hardware while efficiently managing system resources. By providing services such as program execution, I/O management, file handling, communication, security, and resource allocation, it ensures convenient, reliable, and efficient operation of the computer system.



2. What is a System Call? Explain the types of system calls.

Definition of System Call

System calls provide an interface between user programs and the services offered by the operating system. They enable a user program to request services from the operating system kernel.

System calls are generally available as routines written in C or C++ and may also be invoked using assembly-language instructions. Each operating system has its own set of system calls.

Application programmers usually access system calls through an Application Programming Interface (API). The API functions internally invoke the required system calls.

Example: The Windows API function `CreateProcess()` invokes the `NTCreateProcess()` system call in the Windows kernel.

Types of System Calls

System calls are grouped into six major categories:

1. Process Control
2. File Management
3. Device Management
4. Information Maintenance
5. Communication
6. Protection

1. Process Control System Calls

These system calls are used to create, execute, and terminate processes.

Examples:

- `create_process()`
- `terminate_process()`
- `load()`
- `execute()`
- `end()`
- `abort()`
- `wait_event()` After creating new processes, it may need to wait for a certain amount of time to pass to finish its execution.
- `signal_event()` The jobs or processes signals when an event has occurred.
- `get_process_attributes()`
- `set_process_attributes()` are used to control execution of a process.
- `acquire_lock()`
- `release_lock()`

2. File Management System Calls

These calls perform operations on files and directories.

Examples:

- `create()`
- `delete()`
- `open()`
- `close()`
- `read()`
- `write()`
- `reposition()`
- `get_file_attributes()`
- `set_file_attributes()`

3. Device Management System Calls

These calls manage hardware devices and allocate resources.

Examples:

- `request()`
- `release()`
- `read()`
- `write()`
- `reposition()`

4. Information Maintenance System Calls

These calls exchange information between user programs and the operating system.

Examples:

- `get_time()`
- `get_date()`
- `dump()`

They also provide information such as:

- Number of current users
- Operating system version
- Available memory
- Available disk space

5. Communication System Calls

These calls enable inter-process communication using either message passing or shared memory.

Message Passing:

- `gethostid()`
- `getprocessid()`
- `open_connection()`
- `close_connection()`
- `accept_connection()`

Shared Memory:

- `shared_memory_create()`
- `shared_memory_attach()`

6. Protection System Calls

These calls control access to system resources.

Examples:

- `set_permission()`
- `get_permission()`
- `allow_user()`
- `deny_user()`

Protection mechanisms ensure that only authorized users and processes can access files, memory, disks, and other resources.

Conclusion

System calls act as the interface between user programs and the operating system kernel. They allow user applications to request operating system services safely and efficiently. The six major categories of system calls are process control, file management, device management, information maintenance, communication, and protection.

3. What is a System Call? Explain system call implementation with diagram and system call parameter passing.

Definition of System Call

A system call provides an interface between a user program and the services offered by the operating system. It enables a user program to request services from the operating system kernel.

System calls are generally written in C or C++ and may also be invoked using assembly-language instructions. Each operating system provides its own set of system calls.

Application programmers usually access system calls through an Application Programming Interface (API). The API functions internally invoke the required system calls.

Example: The Windows API function CreateProcess() invokes the NtCreateProcess() system call in the Windows kernel.

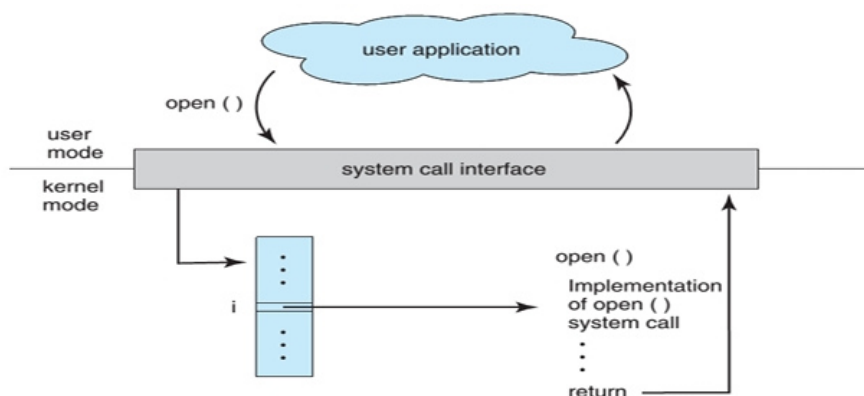
System Call Implementation

For most programming languages, the run-time support system provides a System Call Interface (SCI) that acts as a link between the API and the operating system.

The steps involved in system call implementation are:

1. The application program calls an API function.
2. The API invokes the system call interface.
3. Each system call is associated with a unique number.
4. The system call interface uses this number to identify the required system call from the system call table.
5. Control transfers to the operating system kernel.
6. The kernel executes the requested service.
7. The result and status are returned to the user program through the system call interface.

Diagram of System Call Implementation



user application → API function(eg:open()) → system call interface → system call table → operating system kernel → requested service done → return value

System Call Parameter Passing

When a system call is executed, the operating system needs parameters such as file names, addresses, buffer sizes, or device identifiers.

Parameters can be passed to the operating system in three ways:

1. Passing Parameters in Registers

- Parameters are placed directly into CPU registers before executing the system call.
- This method is simple and efficient.
- However, the number of parameters is limited by the number of available registers.

2. Passing Parameters through a Memory Block or Table

- Parameters are stored in a block or table in memory.
- The address of the block is passed as a parameter in a register.
- This method supports a large number of parameters.

3. Passing Parameters through the Stack

- Parameters are pushed onto the stack by the program.
- The operating system pops the parameters from the stack.
- This method does not limit the number or length of parameters.

Conclusion

System calls provide a controlled interface through which user programs request services from the operating system. The system call interface acts as an intermediary between APIs and the kernel, while parameter passing mechanisms ensure that the required information is transferred efficiently between user space and kernel space.

4. Explain different types of Operating Systems.

An Operating System is system software that manages computer hardware and software resources and provides services to users and application programs.

The major types of operating systems are:

1. Batch Operating System

- Similar jobs are grouped into batches and executed one after another without user interaction.

- Users submit jobs to the operator, and the system processes them automatically.
- Suitable for large and repetitive tasks.

Advantages:

- Improves throughput - amount of work a computer does in certain amount of time
- Reduces setup time

Disadvantages:

- No direct user interaction
- Long turnaround time

Examples: Payroll processing, bank statement generation.

2. Multiprogramming Operating System

- Multiple programs are kept in memory simultaneously.
- When one process waits for I/O, the CPU switches to another process.
- This improves CPU utilization.

Advantages:

- Better CPU utilization
- Increased throughput

Disadvantages:

- Complex memory management
- Requires CPU scheduling

3. Time-Sharing Operating System

- Multiple users share system resources simultaneously.
- Each user gets a small time slice of CPU time.
- The CPU switches rapidly among users, giving the illusion that each user has a dedicated system.

Advantages:

- Quick response time
- Supports multiple users

Disadvantages:

- High overhead
- Security concerns

Examples: UNIX, Linux.

4. Multiprocessing Operating System

- Uses two or more processors that share memory and other resources.
- Multiple processes execute simultaneously.

Advantages:

- Increased throughput
- Improved reliability
- Better resource sharing

Disadvantages:

- Complex design
- Expensive hardware

Types:

- Symmetric Multiprocessing (SMP)
- Asymmetric Multiprocessing (AMP)

5. Distributed Operating System

- A collection of independent computers connected through a network appears as a single system to users.

Advantages:

- Resource sharing
- Improved reliability
- Increased computation speed

Disadvantages:

- Network dependency
- Complex implementation

6. Network Operating System

- Provides services such as file sharing, printer sharing, and communication over a network.

Advantages:

- Centralized administration
- Easy resource sharing

Disadvantages:

- Server dependency
- Higher maintenance cost

Examples: Windows Server, Novell NetWare.

7. Real-Time Operating System (RTOS)

- Used where tasks must be completed within strict time constraints.
- Failure to respond within the specified time may lead to system failure.

Types:

- Hard Real-Time System
- Soft Real-Time System

Applications:

- Industrial control systems
- Medical imaging systems
- Automobile engine control systems
- Weapon systems

8. Embedded Operating System

- Designed for dedicated devices with specific functions.
- Usually operates with limited memory and processing power.

Examples:

- Printers
- Microwave ovens
- GPS devices
- Digital cameras

9. Mobile Operating System

- Designed for smartphones and tablet computers.
- Optimized for portability, low power consumption, and touch interfaces.

Examples:

- Android
- iOS

Conclusion

Different types of operating systems are designed to meet different computing requirements. Batch systems focus on throughput, time-sharing systems support multiple users, distributed systems enable resource sharing, while real-time and embedded systems are designed for specialized applications. Choosing an operating system depends on the application's requirements and available resources.

5. What is Interrupt-Driven I/O Cycle? Explain with the help of a diagram.

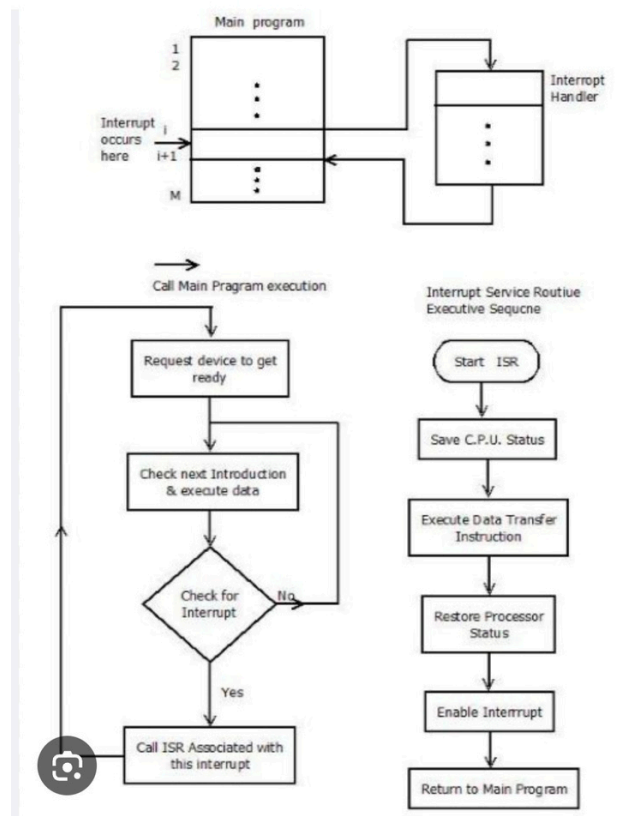
Interrupt-Driven I/O

Interrupt-driven I/O is an I/O mechanism in which the CPU initiates an I/O operation and continues executing other instructions instead of waiting for the I/O operation to complete. When the I/O operation is completed, the device controller sends an interrupt signal to the CPU. The CPU then executes the appropriate Interrupt Service Routine (ISR). This mechanism improves CPU utilization and overall system efficiency.

Working of Interrupt-Driven I/O Cycle

1. The user program requests an I/O operation.
2. The operating system invokes the appropriate device driver.
3. The device driver loads the required registers of the device controller with commands such as read or write.
4. The device controller starts the I/O operation and transfers data between the device and its local buffer.
5. Meanwhile, the CPU continues executing other processes.
6. After completing the I/O operation, the device controller sends an interrupt signal to the CPU.
7. The CPU suspends its current activity and transfers control to the Interrupt Service Routine (ISR).
8. The ISR processes the interrupt, updates the status information, and returns control to the interrupted process.

Diagram of Interrupt-Driven I/O Cycle-(in the diagram not introduction,its interruption)



- The addresses of all Interrupt Service Routines are stored in the Interrupt Vector Table.
- Interrupt-driven I/O improves CPU utilization by eliminating busy waiting.

Advantages of Interrupt-Driven I/O

1. Eliminates busy waiting.
2. Improves CPU utilization.
3. Allows overlap of computation and I/O operations.
4. Increases overall system performance and efficiency.

Conclusion

Interrupt-driven I/O allows the CPU to perform useful work while I/O operations are in progress. The device controller notifies the CPU through interrupts after completing the I/O operation, thereby improving resource utilization and system performance.

6. Define System Program. List and explain different types of System Programs.

Definition of System Program

System programs, also known as system utilities, provide a convenient environment for program development and execution. They are associated with the operating system but are not necessarily part of the kernel.

System programs make the computer system more convenient to use by providing common services to users and programmers.

Types of System Programs

1. File Management Programs

These programs perform various operations on files and directories.

Functions include:

- Create files and directories
- Delete files and directories
- Copy files
- Rename files
- Print files
- List file contents
- Manipulate files and directories

2. Status Information Programs

These programs provide information about system performance and status.

Functions include:

- Display date and time
- Show available memory and disk space
- Display number of users
- Provide performance statistics
- Generate logging and debugging information

3. File Modification Programs

These programs are used to **create, edit, search, and modify file contents.**

Examples include:

- Text editors
- Search utilities
- File update tools

4. Programming Language Support Programs

These programs help users **develop and execute application software.**

Examples include:

- Compilers
- Assemblers
- Interpreters
- Debuggers

They support programming languages such as C, C++, and Java.

5. Program Loading and Execution Programs

These programs **load executable files into memory and prepare them for execution.**

Examples include:

- Loaders
- Linkage editors
- Overlay loaders

6. Communication Programs

These programs **provide communication facilities between users, processes, and computer systems.**

Functions include:

- Sending messages
- Browsing web pages
- Sending e-mails
- Remote login
- File transfer between systems

7. Background Services

Some system programs **run continuously in the background after system boot.**

These are known as:

- Services
- Subsystems
- Daemons

They perform system-related tasks and continue executing until the system is shut down.

Conclusion

System programs provide a user-friendly environment for program development and execution. They simplify file management, communication, programming, and system monitoring, thereby enhancing the usability and efficiency of the operating system.

7. Explain Layered Architecture and Monolithic Architecture. Write the advantages and disadvantages of both architectures.

Operating systems use different structures to organize their components and services. Two common operating system architectures are Monolithic Architecture and Layered Architecture.

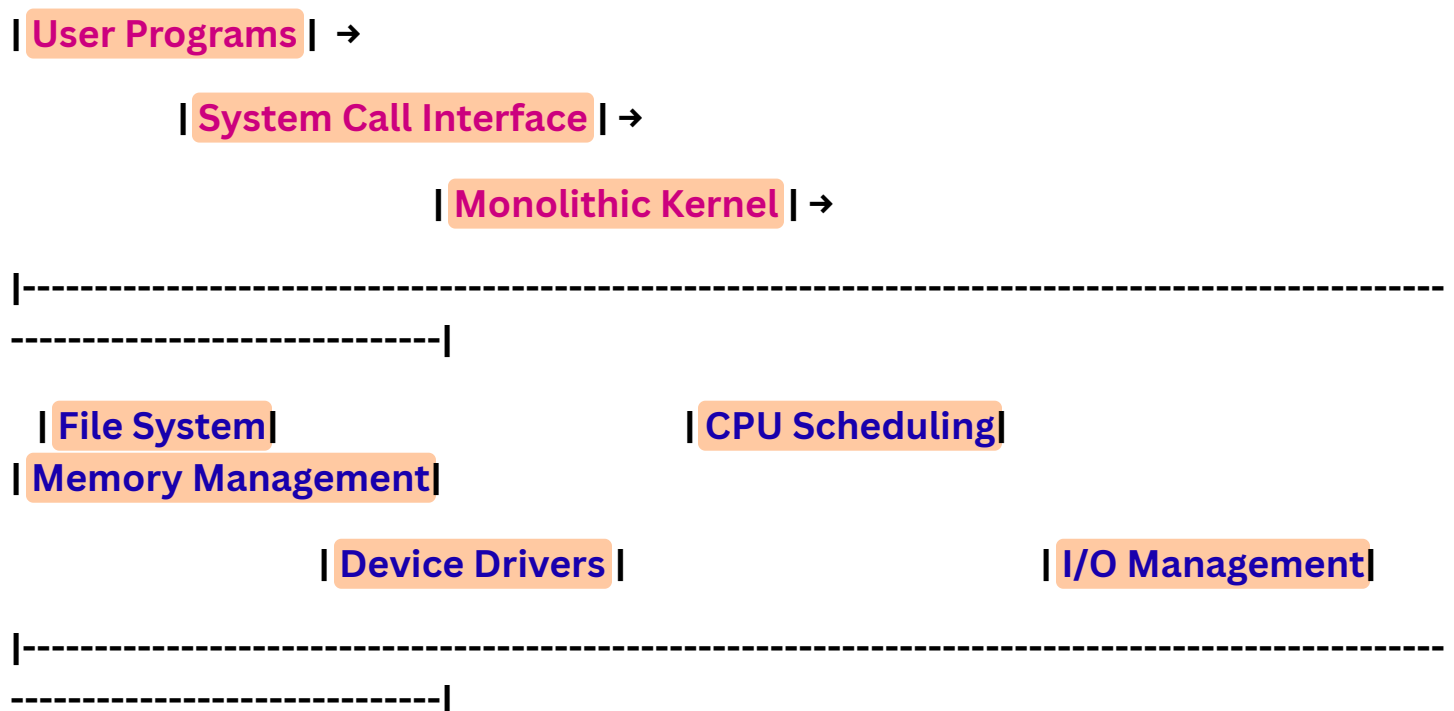
Monolithic Architecture

In a monolithic architecture, the entire operating system works as a single large program running in kernel mode.

All operating system services such as file management, memory management, CPU scheduling, device drivers, and I/O management are combined into one kernel. The traditional UNIX operating system follows a monolithic structure.

Everything below the system-call interface and above the physical hardware is the kernel. The kernel provides file system management, CPU scheduling, memory management, and other operating-system functions through system calls.

Diagram of Monolithic Architecture



Advantages of Monolithic Architecture

1. High performance due to direct communication among components.
2. Efficient execution because all services run in kernel space.
3. Low overhead compared to layered systems.

Disadvantages of Monolithic Architecture

1. Difficult to implement and maintain.
2. Poor modularity.
3. Difficult to debug and extend.
4. Failure of one component may crash the entire system.

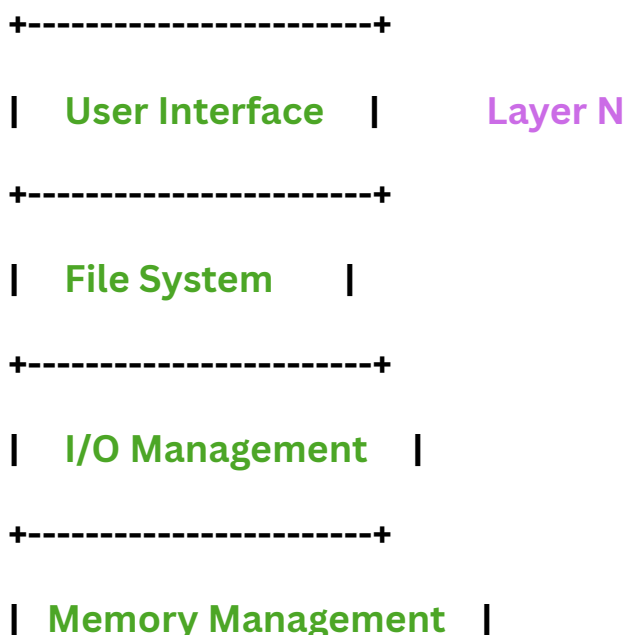
Layered Architecture

In the layered approach, the operating system is divided into a number of layers or levels.

- Layer 0 represents the hardware.
- The highest layer represents the user interface.
- Each layer uses only the services of the layer immediately below it.

Each operating system layer consists of data structures and a set of routines that can be invoked by higher-level layers and can invoke operations of lower-level layers.

Diagram of Layered Architecture



+-----+

| CPU Scheduling |

+-----+

| Hardware | Layer 0

+-----+

Advantages of Layered Architecture

1. Simplicity of construction and debugging.
2. Easier system verification.
3. Better modularity.
4. Easy maintenance and modification.
5. Provides data hiding because each layer hides its implementation details from higher layers.

Disadvantages of Layered Architecture

1. Difficult to define appropriate layers.
2. Careful planning is required because a layer can use only lower-level layers.
3. Less efficient due to overhead at each layer.
4. System calls take more time because parameters and data pass through multiple layers.

Comparison between Monolithic and Layered Architecture

Feature	Monolithic Architecture	Layered Architecture
Structure	Single large kernel	Multiple layers
Modularity	Low	High
Performance	High	Lower due to overhead
Maintenance	Difficult	Easy
Debugging	Difficult	Easy
Reliability	Lower	Higher
Extensibility	Difficult	Easy

Conclusion

Monolithic architecture provides better performance because all operating system services are integrated into a single kernel. However, it is difficult to maintain and debug. Layered architecture improves modularity, maintainability, and debugging by organizing the operating system into multiple layers, although it introduces additional overhead. The choice of architecture depends on system requirements and design goals.

8. Explain the microkernel approach to system design. How do user programs and system services interact in a microkernel architecture? What are the advantages and disadvantages of using the microkernel approach?

Microkernel Approach

As operating systems such as UNIX expanded, their kernels became large and difficult to manage. To overcome this problem, researchers at Carnegie Mellon University developed the Mach operating system, which introduced the microkernel approach.

In the microkernel approach, all nonessential components are removed from the kernel and implemented as user-level programs, resulting in a smaller

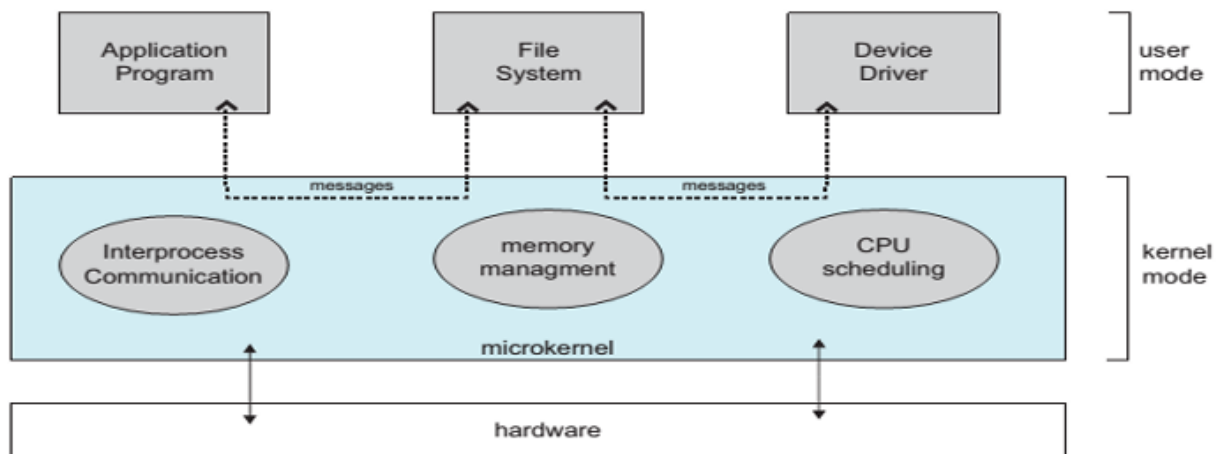
kernel called a microkernel.

The microkernel provides only essential services such as:

- Process management
- Memory management
- Interprocess communication (IPC) using message passing

Other services such as file systems, device drivers, and network services run in user space.

Microkernel Architecture



Interaction Between User Programs and System Services

The main function of the microkernel is to provide message-passing communication between client programs and system services running in user space.

The interaction takes place as follows:

1. A user program requests a service.
2. The request is sent to the microkernel through a message.
3. The microkernel forwards the message to the appropriate server, such as the file server or device server.
4. The server processes the request.
5. The result is sent back to the microkernel.
6. The microkernel returns the response to the user program.

For example, when a client program needs to access a file, it communicates with the file server indirectly through the microkernel. The client and server never interact directly.

Advantages of Microkernel

- Easier to extend the operating system.
- New services can be added in user space without modifying the kernel.
- Easier to port the operating system to different hardware platforms.
- Improved security and reliability because most services run as user processes.
- Failure of one service does not affect the entire operating system.
- Smaller kernel is easier to maintain and debug.

Disadvantages of Microkernel

- Performance may decrease due to increased communication overhead.
- Frequent message passing between user space and kernel space increases execution time.

Conclusion

The microkernel approach structures the operating system by keeping only essential services inside the kernel and moving all other services to user space. Communication between user programs and system services is achieved through message passing. Although microkernels provide better modularity, security, and reliability, they may suffer from performance overhead due to frequent interprocess communication.

9. What are the advantages of designing an operating system using a virtual machine architecture?

Virtual Machine Architecture

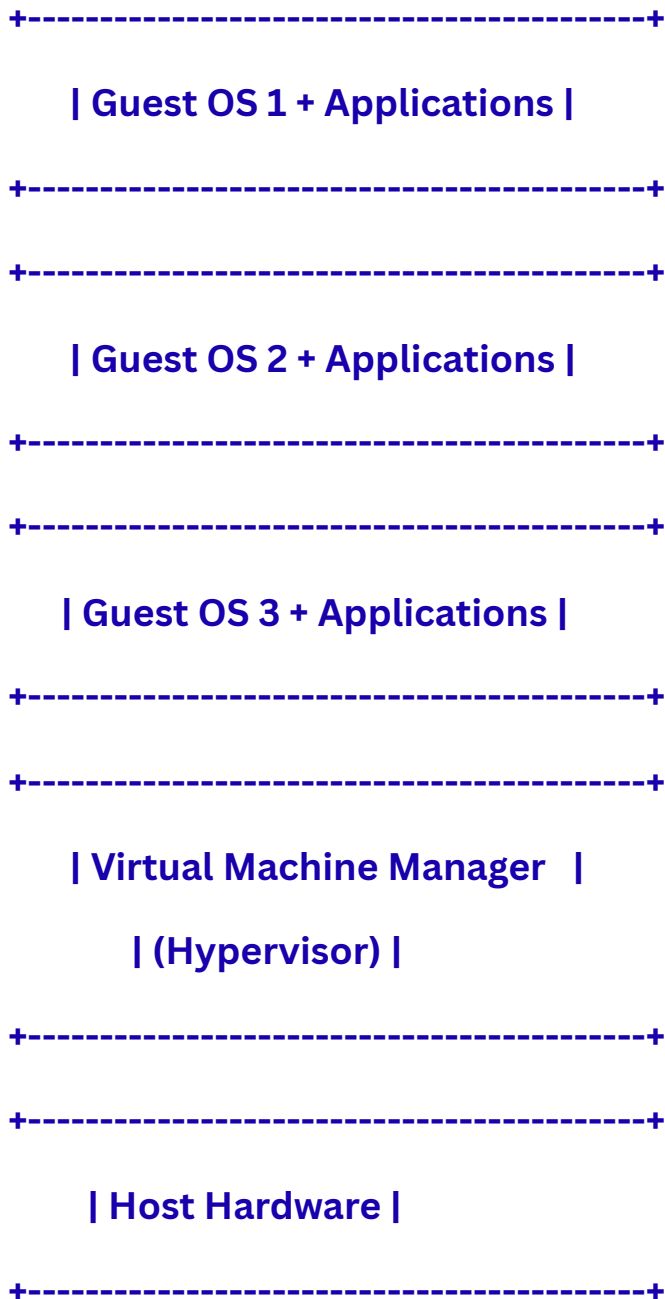
Virtualization is a technology that allows operating systems to run as applications within other operating systems. It enables an entire operating system written for one platform to run on another platform.

A software layer called the Virtual Machine Manager (VMM) or Hypervisor manages virtual machines.

- The original operating system is called the Host Operating System.
- The operating systems running inside virtual machines are called Guest Operating Systems.

Example: An Apple laptop running macOS can run a Windows guest operating system using VMware. Here, macOS is the host OS, VMware is the VMM, and Windows is the guest OS.

Virtual Machine Architecture Diagram



Advantages of Virtual Machine Architecture

1. Better Resource Utilization

Multiple virtual machines can run simultaneously on a single physical machine, leading to efficient utilization of CPU, memory, and storage resources.

2. Isolation and Security

Each virtual machine operates independently. Failure or security issues in one virtual machine do not affect other virtual machines.

3. Support for Multiple Operating Systems

Different operating systems can run concurrently on the same hardware platform.

For example, Linux and Windows can run simultaneously on one computer.

4. Easy Testing and Development

Developers can test new applications and operating systems without affecting the host system.

5. Simplified System Maintenance

Virtual machines can be easily created, copied, backed up, restored, or deleted.

6. Hardware Independence

Virtual machines provide a uniform hardware interface, making it easier to migrate applications between different hardware platforms.

7. Improved Reliability

Since virtual machines are isolated from each other, failure of one guest operating system does not crash the entire system. The VMM protects each guest from the others.

8. Cost Reduction

Running multiple virtual machines on a single physical system reduces hardware, power, and maintenance costs.

Disadvantages of Virtual Machine Architecture

- **Additional overhead due to virtualization.**
- **Performance may be lower than native execution.**
- **Requires sufficient hardware resources.**

Conclusion

Virtual machine architecture allows multiple operating systems to run simultaneously on a single physical machine by using a Virtual Machine Manager. It provides better resource utilization, security, reliability, portability, and cost savings, making virtualization an important technology in modern computing environments.

Unit 1 – SAQs

1. What is an Operating System?

An Operating System (OS) is system software that acts as an intermediary between the user and computer hardware. It provides an environment for executing programs conveniently and efficiently. It also acts as a resource allocator and control program.

2. What are the services of an Operating System?

The major services provided by an operating system are:

- 1. User Interface (CLI, GUI, Batch)**
- 2. Program Execution**
- 3. I/O Operations**
- 4. File System Manipulation**
- 5. Communication**
- 6. Error Detection**
- 7. Resource Allocation**
- 8. Accounting**
- 9. Protection and Security**

3. Explain the view of Operating System as a Resource Manager.

Operating system acts as a resource manager by allocating and managing resources such as CPU time, memory space, file storage, and I/O devices among users and processes efficiently and fairly. It decides which resource is allocated to which process and for how long.

4. What are the five major activities of an operating system with regard to file management?

The operating system performs the following file management activities:

- 1. Create and delete files.**
- 2. Create and delete directories.**
- 3. Read and write files.**
- 4. Search files and directories.**
- 5. Manage file permissions and ownership.**

5. Explain the concept of dual mode operations of operating system:

a) Kernel Mode

b) User Mode

Operating systems support dual-mode operation to protect the system from errant users and programs.

- **User Mode:** The system executes user applications. Privileged instructions cannot be executed in this mode.
- **Kernel Mode:** The operating system executes privileged instructions and has complete access to hardware resources.

A mode bit indicates the current mode:

- Kernel mode = 0
- User mode = 1

Whenever a system call, trap, or interrupt occurs, the system switches from user mode to kernel mode.

6. Explain the usage of timer or counter by operating system.

A timer is used to maintain control over the CPU and prevent infinite loops.

- The operating system sets a timer before transferring control to a user process.
- Every clock tick decrements the counter.
- When the counter reaches zero, an interrupt occurs.
- Control is transferred back to the operating system.

Instructions that modify the timer are privileged instructions.

7. Draw the diagram of storage structure hierarchy.

Draw the following hierarchy:



|

Magnetic Disk

|

Optical Disk/Tape

As we move downward:

- Access speed decreases.
- Cost per bit decreases.
- Capacity increases.

8. Draw the structure of operating system.



The operating system acts as an interface between application programs and hardware.

9. Explain operating system generations.

1. **First Generation (1945–1955):**
 - Vacuum tubes
 - No operating system
2. **Second Generation (1955–1965):**
 - Transistors
 - Batch operating systems
3. **Third Generation (1965–1980):**
 - Integrated circuits
 - Multiprogramming and time-sharing systems
4. **Fourth Generation (1980–Present):**
 - Personal computers
 - GUI-based operating systems
5. **Fifth Generation:**
 - Parallel processing
 - Distributed systems
 - Artificial intelligence concepts

10. What is a CLI?

Command Line Interface (CLI) allows users to directly enter commands to be executed by the operating system.

The command interpreter in UNIX and Linux is called a shell.

Examples: Bourne shell, C shell, Korn shell, Bash shell.

11. Define and distinguish system call and system program.

System Call	System Program
Interface between user program and operating system.	Utilities that provide a convenient environment for program development and execution.
Invokes kernel services.	Uses system calls to perform tasks.
Executed in kernel mode.	Executed in user mode.
Example: open(), read(), write()	Example: Compiler, editor, file manager

12. Define the concept of Virtual Machines.

A virtual machine is an environment that allows an operating system to run as an application within another operating system.

The software that manages virtual machines is called the Virtual Machine Manager (VMM) or Hypervisor.

The original operating system is called the Host OS, and the operating systems running inside virtual machines are called Guest OSs.

Example: VMware running Linux as a guest operating system on Windows.

Add these extra topics ★

1. Objectives/Goals of an Operating System

User goals:

- Convenient to use
- Easy to learn
- Reliable
- Safe
- Fast

System goals:

- Easy to design and maintain
- Flexible
- Efficient
- Error-free
- Reliable

2. Mechanism vs Policy

- Mechanism: Determines *how* something is done.
- Policy: Determines *what* will be done.

Example:

- Timer → Mechanism
- Deciding timer duration → Policy

This is a favorite short question.

3. API vs System Call

API	System Call
Interface used by programmers	Interface between program and OS
Higher level	Lower level
Invokes system calls internally	Executes kernel services

Example:

CreateProcess() → API

NTCreateProcess() → System Call

4. Bootstrap Program

- Stored in ROM.
- Initializes CPU registers, device controllers, and memory.
- Locates and loads the operating system kernel into memory.

5. Privileged Instructions

Instructions that can execute only in kernel mode.

Examples:

- I/O control
- Timer management
- Interrupt management

6. Host OS, Guest OS, VMM

For virtual machines, remember these terms:

- Host OS: Original operating system.
- Guest OS: Operating system running inside VM.
- VMM/Hypervisor: Manages virtual machines.

7. CLI Command Execution Methods

Commands are implemented in two ways:

1. Command interpreter contains the command code.
2. Command interpreter loads and executes an external file.

Example:

`rm file.txt` in UNIX.